Lekcja 2: Wprowadzenie do ekosystemu Python

`#lekcja` `#python` `#ekosystem` `#narzędzia` `#linux`

Na tej lekcji zanurzymy się głębiej w świat Pythona. Dowiemy się, co czyni ten język tak wyjątkowym, poznamy jego "filozofię", skonfigurujemy profesjonalne środowisko pracy za pomocą maszyny wirtualnej i oczywiście napiszemy nasze pierwsze sensowne programy.

1. Pierwsze spotkanie z Pythonem

Python to **wysokopoziomowy język programowania ogólnego przeznaczenia**. Rozłóżmy to na czynniki pierwsze:

- **Wysokopoziomowy:** Nie musisz myśleć o operacjach niskiego poziomu, takich jak zarządzanie pamięcią czy praca z rejestrami procesora. Możesz skupić się na logice rozwiązania zadania.
- **Ogólnego przeznaczenia:** Można w nim tworzyć praktycznie wszystko: strony internetowe, sieci neuronowe, obliczenia naukowe, gry, skrypty do automatyzacji i wiele więcej.

> **Info**
>
> Język został stworzony przez holenderskiego programistę Guido van Rossuma i po raz pierwszy wydany w 1991 roku.

Dlaczego Python jest tak popularny?

- **Niski próg wejścia:** Łatwo jest zacząć się go uczyć dzięki prostej i czystej składni.
- **Ogromna i aktywna społeczność:** Jeśli napotkasz problem, najprawdopodobniej ktoś już się z nim zmierzył i rozwiązanie jest łatwe do znalezienia.
- **Potężna biblioteka standardowa i tysiące pakietów zewnętrznych:** Dla większości zadań istnieją już gotowe rozwiązania.

> **Tip**
>
> Nazwa języka nie pochodzi od węża 🐍, ale od brytyjskiego serialu komediowego "Latający cyrk Monty Pythona" (Monty Python's Flying Circus).

2. Podstawowe cechy języka

- **Prosta i czytelna składnia:** Zamiast złożonych konstrukcji i nawiasów klamrowych, Python używa wcięć do definiowania bloków kodu. Zmusza to do pisania ustrukturyzowanego i łatwego do czytania kodu. **Porównaj:**

```
// Java
for (int i = 0; i < 5; i++) {
    System.out.println("Hello!");
}
```

```
# Python
for i in range(5):
    print("Hello!")
```

- **Język interpretowany:** Kod jest wykonywany linijka po linijce przez specjalny program — interpreter, co upraszcza debugowanie. Nie musisz kompilować programu przed uruchomieniem.
- **Dynamiczne typowanie:** Nie musisz deklarować typu zmiennej. Python sam go określa w trakcie wykonania programu.

```
x = 10          # x jest liczbą (int)
x = "Hello"     # Teraz x jest ciągiem znaków (str)
```

- **Wieloplatformowość:** Kod napisany na Windowsie bez zmian uruchomi się na macOS lub Linuksie (i odwrotnie).
- **Ogromny ekosystem bibliotek:** Za pomocą menedżera pakietów `pip` można zainstalować tysiące darmowych bibliotek do dowolnych zadań:
  - `NumPy` & `Pandas` do analizy danych.
  - `TensorFlow` & `PyTorch` do uczenia maszynowego.
  - `Django` & `Flask` do tworzenia stron internetowych.
  - `Requests` do pracy z siecią.
  - `Pygame` do tworzenia gier.

3. Porównanie Pythona z innymi językami

Cecha	Python	C/C++	Java	JavaScript
Składnia	Bardzo prosta, minimalistyczna	Złożona, wymaga ręcznego zarządzania pamięcią	Bardziej rozwlekła, ścisła	Podobna do C/Javy, ale z elastycznością
Paradygmat	Obiektowy, proceduralny,	Głównie proceduralny (C)	Ściśle obiektowy	Wieloparadygmatowy, zorientowany na

	funkcyjny	i obiektowy (C++)		zdarzenia
Typowanie	Dynamiczne	Statyczne	Statyczne	Dynamiczne
Wydajność	Umiarkowana	Bardzo wysoka (blisko "sprzętu")	Wysoka (dzięki JVM)	Wysoka (w nowoczesnych silnikach)
Główne zastosowanie	Web (backend), Data Science, AI, skrypty	Oprogramowanie systemowe, gry, sterowniki, obliczenia o wysokiej wydajności	Aplikacje korporacyjne, Android, duże systemy	Web (frontend, backend), aplikacje mobilne
Próg wejścia	Bardzo niski	Wysoki	Średni	Średni

4. Zen Pythona (The Zen of Python)

> **Quote**
>
> "Zen Pythona" to zbiór 19 zasad i aforyzmów, które wpływają na projekt języka Python. To jego filozofia.

Aby je zobaczyć, po prostu wykonaj w interpreterze Pythona:

```
import this
```

Niektóre z kluczowych zasad:

- Piękne jest lepsze niż brzydkie.
- Jawne jest lepsze niż domniemane.
- Proste jest lepsze niż złożone.
- Złożone jest lepsze niż skomplikowane.
- Czytelność ma znaczenie.

Te zasady zachęcają do tworzenia prostego, zrozumiałego i łatwego w utrzymaniu kodu.

5. Konfiguracja środowiska pracy: VirtualBox i Ubuntu na Windows

Do poważnego programowania często używa się Linuksa. Jest szybszy, stabilniejszy, a większość serwerów w internecie działa właśnie na nim. Aby nie usuwać systemu Windows, możemy zainstalować Linuksa на **maszynie wirtualnej** — to "komputer wewnątrz komputera".

Po co to robić?

- **Środowisko jak na serwerze:** Twój kod będzie działał w tym samym środowisku, w którym będzie "żyć" w przyszłości.
- **Lepsze narzędzia:** Wiele narzędzi deweloperskich działa lepiej lub istnieje tylko pod Linuksem.
- **Izolacja:** Nie "zaśmiecasz" swojego głównego systemu Windows eksperymentami.

Krok 1: Instalacja VirtualBox

1. Przejdź na oficjalną stronę: virtualbox.org.
2. Pobierz pakiet instalacyjny dla **Windows hosts**.
3. Uruchom instalator i postępuj zgodnie z instrukcjami, pozostawiając wszystkie ustawienia domyślne.

Krok 2: Pobieranie obrazu Ubuntu

1. Przejdź na oficjalną stronę Ubuntu: ubuntu.com/download/desktop.
2. Pobierz najnowszą wersję **LTS (Long-Term Support)**. Jest to najbardziej stabilna wersja z długoterminowym wsparciem. Pobierzesz plik z rozszerzeniem `.iso`.

Krok 3: Tworzenie maszyny wirtualnej

1. Otwórz VirtualBox i kliknij przycisk "Nowa" (niebieska gwiazda).
2. **Nazwa i obraz:**
   - **Nazwa:** Wpisz `Ubuntu Desktop`.
   - **Folder:** Pozostaw domyślny.
   - **Obraz ISO:** Kliknij ikonę i wybierz pobrany plik `.iso` z Ubuntu.
   - Zaznacz pole **"Pomiń instalację nienadzorowaną"**.
3. **Sprzęt:**
   - **Pamięć RAM:** Przydziel co najmniej `4096 MB` (4 GB). Jeśli masz dużo RAM (16 GB lub więcej), możesz przydzielić 8 GB.
   - **Procesory (CPU):** Przydziel 2 lub 4 rdzenie.
4. **Wirtualny dysk twardy:**
   - Wybierz **"Stwórz nowy wirtualny dysk twardy teraz"**.
   - Rozmiar dysku: Ustaw **`40 GB`** lub więcej.
5. Kliknij "Zakończ".

Krok 4: Instalacja Ubuntu

1. Wybierz swoją maszynę `Ubuntu Desktop` z listy i kliknij "Uruchom".
2. Otworzy się okno. W pierwszym menu wybierz **"Try or Install Ubuntu"** i naciśnij Enter.
3. Po załadowaniu pulpitu kliknij ikonę **"Install Ubuntu"**.
4. **Proces instalacji:**
   - **Język:** Wybierz "Polski" lub "English".
   - **Układ klawiatury:** Pozostaw domyślny.
   - **Aktualizacje i oprogramowanie:** Wybierz **"Minimalna instalacja"** i odznacz pole "Pobierz aktualizacje podczas instalacji".
   - **Typ instalacji:** Wybierz **"Wyczyszczenie dysku i zainstalowanie Ubuntu"**.

> ⚠ **Nie bój się! Ta operacja sformatuje tylko wirtualny dysk twardy, który stworzyliśmy, a nie Twój główny dysk z systemem Windows.**

```
- **Strefa czasowa:** Wybierz swoje miasto.

- **Tworzenie użytkownika:** Wpisz swoje imię, nazwę komputera, nazwę
użytkownika (np. `student`) i wymyśl hasło.
```

5. Poczekaj na zakończenie instalacji i uruchom ponownie maszynę wirtualną.

Krok 5: Konfiguracja Ubuntu po instalacji

1. **Instalacja "Dodatków gościa" (Guest Additions):** Jest to bardzo ważne dla normalnej pracy (wspólny schowek, zmiana rozmiaru ekranu).
   - W oknie uruchomionej maszyny Ubuntu, w górnym menu VirtualBox wybierz `Urządzenia -> Zamontuj obraz płyty z Dodatkami gościa...`.
   - Na pulpicie Ubuntu pojawi się ikona dysku. Otwórz terminal (`Ctrl+Alt+T`).
   - Wykonaj polecenia:
     ```
     sudo apt update
     sudo apt install build-essential dkms linux-headers-$(uname -r)
     cd /media/$USER/VBox_GAs_*
     sudo ./VBoxLinuxAdditions.run
     ```
   - Uruchom ponownie maszynę.
2. **Konfiguracja wspólnego schowka:** W ustawieniach maszyny w VirtualBox (`Ustawienia -> Ogólne -> Zaawansowane`) ustaw "Wspólny schowek" i "Przeciąganie i upuszczanie" na **"Dwukierunkowy"**.

Teraz masz pełnoprawne środowisko Linux do programowania!

5a. Konfiguracja środowiska pracy Linux na Windows

Do poważnego programowania często używa się środowiska linuksowego. Zapewnia ono zgodność z większością serwerów i dostarcza potężne narzędzia wiersza poleceń. Dla użytkowników systemu Windows istnieją dwa doskonałe sposoby na uzyskanie dostępu do Linuksa.

Sposób 1: WSL 2 (Podsystem Windows dla Linuksa) - Zalecany

WSL (Windows Subsystem for Linux) — to technologia firmy Microsoft, która pozwala na uruchomienie pełnoprawnego środowiska Linux bezpośrednio w systemie Windows, bez konieczności tworzenia maszyny wirtualnej.

Zalety WSL 2:

- **Wysoka wydajność:** WSL 2 działa na prawdziwym jądrze Linuksa i zapewnia niemal natywną prędkość pracy z plikami i obliczeniami.
- **Ścisła integracja:** Możesz wywoływać polecenia Linuksa z Windows PowerShell i odwrotnie. Pliki obu systemów są łatwo dostępne.
- **Oszczędność zasobów:** Zużywa znacznie mniej pamięci RAM i zasobów procesora niż pełnoprawna maszyna wirtualna.

Instalacja WSL 2

Na nowoczesnych systemach Windows 10/11 jest to zaskakująco proste.

1. Naciśnij `Win + X` i wybierz **"Terminal (Administrator)"** lub **"Windows PowerShell (Administrator)"**.
2. W otwartym oknie wpisz tylko jedno polecenie i naciśnij Enter:
   ```
   wsl --install
   ```
3. To polecenie automatycznie wykona wszystkie niezbędne czynności: włączy odpowiednie komponenty systemu Windows, pobierze jądro Linuksa i domyślnie zainstaluje dystrybucję **Ubuntu**.
4. Po zakończeniu **uruchom ponownie komputer**.
5. Przy pierwszym uruchomieniu Ubuntu (z menu "Start") zostaniesz poproszony o utworzenie nazwy użytkownika i hasła dla Twojego środowiska Linux. **Wymyśl je i zapamiętaj!**

Gratulacje, masz Linuksa wewnątrz systemu Windows! Możesz go uruchamiać jak zwykłą aplikację "Ubuntu" z menu "Start".

Sposób 2: VirtualBox i Lubuntu (Metoda uniwersalna)

Jeśli używasz starszej wersji systemu Windows lub potrzebujesz pełnej izolacji graficznej systemu, maszyna wirtualna pozostaje doskonałym wyborem.

Krok 1: Instalacja VirtualBox

1. Pobierz instalator z oficjalnej strony virtualbox.org dla **Windows hosts**.
2. Zainstaluj program, postępując zgodnie z instrukcjami (można pozostawić wszystkie ustawienia domyślne).

Krok 2: Pobieranie obrazu Lubuntu

Zamiast standardowego Ubuntu, polecamy **Lubuntu** dla maszyny wirtualnej.

> **Co to jest Lubuntu?**
>
> Lubuntu — to oficjalna, "lekka" wersja Ubuntu. Używa środowiska graficznego LXQt, które wymaga znacznie mniej pamięci RAM i zasobów procesora niż standardowe środowisko GNOME w Ubuntu. W rezultacie Lubuntu działa w maszynie wirtualnej zauważalnie szybciej.

1. Przejdź na oficjalną stronę Lubuntu: lubuntu.me/downloads/.
2. Pobierz najnowszą wersję **LTS (Long-Term Support)**. Otrzymasz plik z rozszerzeniem `.iso`.

Krok 3 i 4: Tworzenie maszyny wirtualnej i instalacja Lubuntu

Proces tworzenia maszyny wirtualnej i instalacji Lubuntu jest praktycznie identyczny z instalacją Ubuntu, którą omawialiśmy wcześniej:

1. W VirtualBox kliknij "Nowa".
2. **Nazwa:** `Lubuntu Desktop`.
3. **Obraz ISO:** Wskaż pobrany plik `.iso` Lubuntu.
4. **Sprzęt:** Przydziel **2048 MB (2 GB)** pamięci RAM (dla Lubuntu to wystarczy) i 2 rdzenie procesora.
5. **Dysk twardy:** Utwórz nowy wirtualny dysk o rozmiarze **25-40 GB**.
6. Uruchom maszynę i postępuj zgodnie z instrukcjami instalatora Lubuntu. Jest on bardzo podobny do instalatora Ubuntu. Pamiętaj, aby wybrać **"Wyczyszczenie dysku i zainstalowanie Lubuntu"**.

7. Po instalacji koniecznie zainstaluj **"Dodatki gościa" (Guest Additions)** (`Urządzenia -> Zamontuj obraz płyty...`) dla lepszej integracji.

6. Izolowane środowiska: `venv`

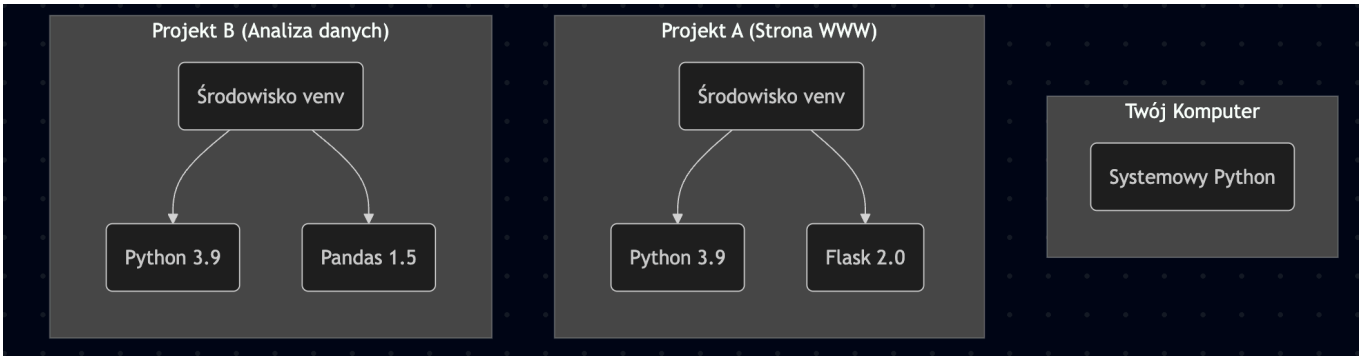
> **Note**
>
> Środowisko wirtualne — to izolowana kopia Pythona z własnym zestawem bibliotek dla konkretnego projektu.

Analogia: Wyobraź sobie, że do naprawy samochodu masz jedną skrzynkę z narzędziami, a do naprawy hydrauliki — zupełnie inną. Nie mieszają się one. `venv` to właśnie taka osobna skrzynka dla każdego Twojego projektu.

```
graph TD
    subgraph "Twój Komputer"
        A("Systemowy Python")
    end

    subgraph "Projekt A (Strona WWW)"
        B("Środowisko venv") --> C("Python 3.9")
        B --> D("Flask 2.0")
    end

    subgraph "Projekt B (Analiza danych)"
        E("Środowisko venv") --> F("Python 3.9")
        E --> G("Pandas 1.5")
    end
```



Jak tego używać? (Polecenia dla Linux/macOS)

```
# Przechodzimy do folderu projektu
cd moj_projekt
```

```
# 1. Tworzymy środowisko o nazwie 'venv'
python3 -m venv venv

# 2. Aktywujemy je
source venv/bin/activate
# Po aktywacji na początku wiersza poleceń pojawi się (venv)

# Teraz możemy instalować biblioteki tylko dla tego projektu
pip install requests

# 3. Aby wyjść ze środowiska
deactivate
```

7. Pierwsze kroki w kodzie

1. Wyświetlanie tekstu na ekranie

Funkcja `print()` wyświetla wszystko, co znajduje się w nawiasach.

```
print("Hello, World!")
print("Python jest fajny.")
```

2. Pobieranie danych od użytkownika

Funkcja `input()` czeka, aż użytkownik wpisze tekst i naciśnie Enter. Zawsze zwraca **ciąg znaków (string)**.

```
# Pytamy użytkownika o imię
imie = input("Jak masz na imię? ")

# Wyświetlamy spersonalizowane powitanie
print("Cześć, " + imie + "!")
```

3. Dodawanie liczb

Aby pracować z danymi od użytkownika jako z liczbami, trzeba je przekonwertować za pomocą `int()` (liczba całkowita) lub `float()` (liczba zmiennoprzecinkowa).

```
# Prosimy o dwie liczby (będą to ciągi znaków)
num1_str = input("Podaj pierwszą liczbę: ")
num2_str = input("Podaj drugą liczbę: ")

# Konwertujemy ciągi znaków na liczby całkowite
```

```
a = int(num1_str)
b = int(num2_str)

# Obliczamy i wyświetlamy sumę
wynik_sumy = a + b
print("Suma:", wynik_sumy)
```

> ⚠️ **Warning**
>
> Jeśli spróbujesz dodać ciągi znaków "5" i "10", otrzymasz nie 15, a "510" (konkatenacja). Zawsze konwertuj typy danych!

🧪 8. Zadania praktyczne

1. **Opowieść o sobie:** Napisz program, który wyświetli na ekranie 3-4 zdania o Tobie (Twoje imię, hobby, cel nauki Pythona). Każde zdanie powinno być wyświetlone osobnym poleceniem `print()`.
2. **Interaktywny dialog:** Napisz program, który zapyta użytkownika o jego imię, wiek i miasto zamieszkania. Następnie wyświetl podsumowanie w jednej linijce, np.: `"A więc, masz на imię [Imię], masz [Wiek] lat i mieszkasz w mieście [Miasto]."`
3. **Prosty kalkulator obwodu:** Napisz program, który poprosi użytkownika o długość i szerokość prostokąta, a następnie obliczy i wyświetli jego obwód (P = 2 * (długość + szerokość)).
4. **Kody znaków:** Używając funkcji `ord()`, znajdź i wyświetl kody liczbowe ASCII dla pierwszej litery Twojego imienia (wielką i małą literą).
5. **Kreator postaci:** Program prosi użytkownika o nazwę rasy (`"Elf"`, `"Krasnolud"` itp.) oraz klasę (`"Wojownik"`, `"Mag"`). Następnie wyświetla: `Stworzono postać: [Rasa] [Klasa].`
6. **Wyrażenie logiczne:** Napisz program, który pyta, czy użytkownik ma prawo jazdy (`tak/nie`) i ile ma lat. Wyświetl `True`, jeśli użytkownik ma 18 lat lub więcej **ORAZ** ma prawo jazdy. W przeciwnym razie wyświetl `False`.
7. **Praktyka z `venv`:**
   - Utwórz folder `project_test`.
   - Wewnątrz niego utwórz środowisko wirtualne `venv`.
   - Aktywuj je.
   - Wykonaj polecenie `pip list`, aby zobaczyć, że nie ma zainstalowanych bibliotek.
   - Dezaktywuj środowisko.
8. **(Dla użytkowników VM) Nawigacja w Ubuntu:**
   - Otwórz terminal w swojej maszynie Ubuntu.

- Utwórz folder `moj_pierwszy_projekt` na pulpicie (`cd Desktop` , a następnie `mkdir moj_pierwszy_projekt`).
- Przejdź do tego folderu.
- Utwórz wewnątrz pusty plik `main.py` poleceniem `touch main.py` .
- Wyświetl zawartość folderu poleceniem `ls` .

9. **Zen w częściach:** Wyświetl na ekranie tylko pierwsze dwie linijki z "Zenu Pythona". *Wskazówka: musisz pobrać cały tekst i przetworzyć go jako ciąg znaków lub listę.*
10. **Mini-projekt "Konwerter Temperatury":** Napisz program, który poprosi użytkownika o temperaturę w stopniach Celsjusza, przekonwertuje ją na stopnie Fahrenheita według wzoru `F = C * 9/5 + 32` i wyświetli wynik z wyjaśnieniem.